Revue & Rétrospective — Sprint 4 : Intégration & Finalisation

1. Revue du sprint — fonctionnalités terminées

- Le **pipeline complet** (pipeline.py) relie la capture Scapy, l'extraction de features, le modèle Isolation Forest et la base du dashboard. Il capture par tranches de 5 secondes, convertit le score du modèle vers l'échelle 0–1 du dashboard, décrit chaque alerte (« Balayage de ports probable… ») et ajoute la réputation AbuseIPDB quand une clé est fournie. Un arrêt par Ctrl+C et les erreurs de capture sont gérés.
- Les **tests d'intégration de bout en bout** (15 tests) font passer de faux paquets dans toute la chaîne, jusqu'à l'API du dashboard, sans réseau ni droits administrateur. Avec les tests du dépôt et du détecteur, la racine du projet compte 27 tests réussis ; le dashboard en compte 44.
- Un **vrai scan de ports** (nmap sur ma propre machine, interface lo0) est détecté de bout en bout et apparaît sur le dashboard avec la raison « Balayage de ports probable : 496 paquets, 494 ports… ».
- **Correction des bugs identifiés** : le modèle déclenchait environ 80 fausses alertes en 70 secondes de navigation normale. Après réentraînement sur 10 minutes de trafic normal (3 438 exemples, contre 34 avant), ajout d'une métrique « ports par paquet » et choix du seuil sur des données non vues à l'entraînement, il reste **2 alertes en 2 min 30** d'usage réel (0,29 % de fausses alertes sur les données de test), et le scan est toujours détecté.
- **Revue finale du code** et **README d'installation** : installation, lancement, mode démonstration, simulation d'attaque, AbuseIPDB, réentraînement, tests, dépannage et limites connues. Ajout de requirements.txt (versions figées) et .gitignore (bases de données et environnement virtuel exclus), nettoyage du dossier, et installation testée avec succès depuis un environnement vide.

Statut : toutes les tâches du Sprint 4 sont terminées.

2. Rétrospective

Ce qui a bien fonctionné

- Les tests d'intégration ont permis de brancher le pipeline avec confiance, et les essais réels (navigation sur le Wi-Fi, scan sur la boucle locale) ont confirmé ce que les tests annonçaient.
- Le premier test en conditions réelles a révélé le problème des fausses alertes, que des tests avec des données fabriquées n'auraient pas montré. Il a été corrigé avec des mesures avant/après plutôt qu'à l'intuition.
- Le seuil de détection est maintenant calculé sur des exemples que le modèle n'a pas vus, puis enregistré avec le modèle : il n'est plus choisi au hasard.

Difficultés rencontrées

- Scapy intercepte lui-même Ctrl+C pendant la capture : le pipeline ne s'arrêtait pas. Corrigé avec un gestionnaire de signal qui termine la tranche en cours, et un test automatique.
- Avec sudo, les variables d'environnement (clé AbuseIPDB) sont perdues et les fichiers créés (base, jeu de données) appartiennent à root. Corrigé en transmettant la clé dans la commande et en rendant les fichiers à l'utilisateur.
- Un modèle appris sur 34 exemples seulement jugeait anormale toute activité un peu soutenue. Corrigé en capturant 10 minutes de trafic varié.

Limites restantes

- Le modèle n'a été entraîné et évalué que sur mon propre trafic et sur des scans nmap (6 exemples de scan au total, 3 pour le test) ; il n'a pas été testé sur un jeu public ni sur d'autres attaques.
- Les 2 fausses alertes restantes sont de gros pics de trafic venant d'une seule adresse (téléchargement ou vidéo).
- Le pipeline entier tourne avec sudo parce que Scapy l'exige, et le dashboard n'a pas d'authentification (il n'écoute que sur 127.0.0.1).

Pistes pour la suite

- Évaluer le modèle sur un jeu de données public étiqueté (par exemple CIC-IDS) et sur d'autres types d'attaques.
- Séparer la capture (administrateur) de l'analyse, ajouter une authentification au dashboard, prendre en charge IPv6 et réentraîner régulièrement le modèle.